Work Placement & Professional

MSc in Software Design with Cloud Native Computing

Design and Evaluation of a Multi-Agent AI System for Incident Diagnosis in Cloud-Native Applications Using Kubernetes Observability Data

Interim Report
(Word count: 550)

Maheendran Padmakumar

A00336089

Technological University of the Shannon: Midlands Midwest

Contents

Introduction

This interim report presents the progress achieved during the initial phase of my Engineering Team Project for the MSc in Software Design with Cloud Native Computing. The project aims to develop a cloud-native microservices platform deployed on Kubernetes and integrate observability technologies to support AI-driven incident diagnosis. Modern cloud-native applications consist of multiple distributed services, making it difficult to identify failures and performance bottlenecks using traditional monitoring techniques. The project therefore focuses on implementing distributed tracing and observability as a foundation for a future multi-agent AI system capable of analysing telemetry data and assisting with root cause analysis.

Responsibilities and Duties

My primary responsibilities during this phase of the project included designing the system architecture, developing independent microservices, containerising applications, deploying services on Kubernetes, and implementing observability components.

The project was developed using a microservices architecture consisting of three independent services: User Service, Payment Service, and Inventory Service. Each service was implemented using FastAPI and containerised using Docker. Kubernetes Deployments and Services were configured to manage application deployment and service discovery within a Minikube cluster. In addition, I researched observability technologies and began integrating OpenTelemetry and Jaeger into the application to enable distributed tracing across the microservices.

Technical Achievements

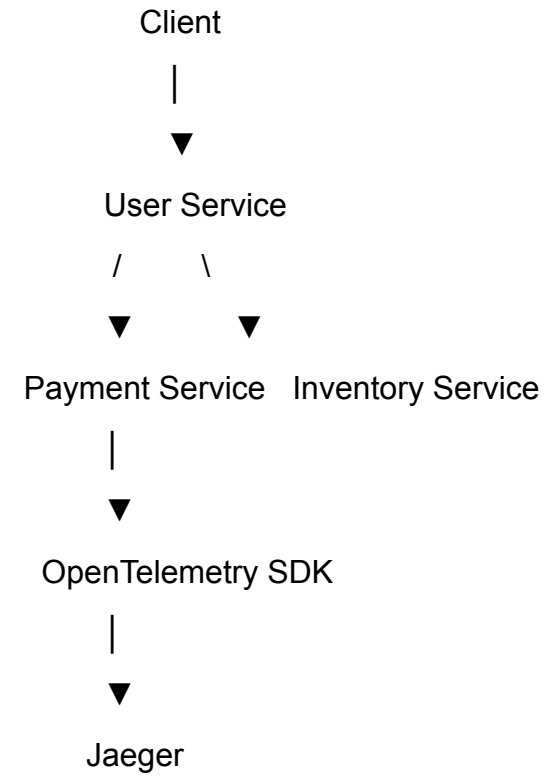
During this reporting period, I successfully developed and deployed three functional microservices. The User Service acts as the entry point for client requests and communicates with both the Payment Service and Inventory Service through REST APIs. Kubernetes Service Discovery was used to enable communication between services without requiring static IP addresses.

The complete application was deployed within a Kubernetes cluster using separate Deployments and Services for each microservice. Docker images were built and loaded into Minikube, allowing the application to run entirely within a cloud-native environment.

A significant achievement during this phase was the successful deployment of Jaeger as the project's distributed tracing platform. OpenTelemetry libraries were installed and configured within the User Service to generate distributed traces. These traces were successfully exported to Jaeger using the OpenTelemetry

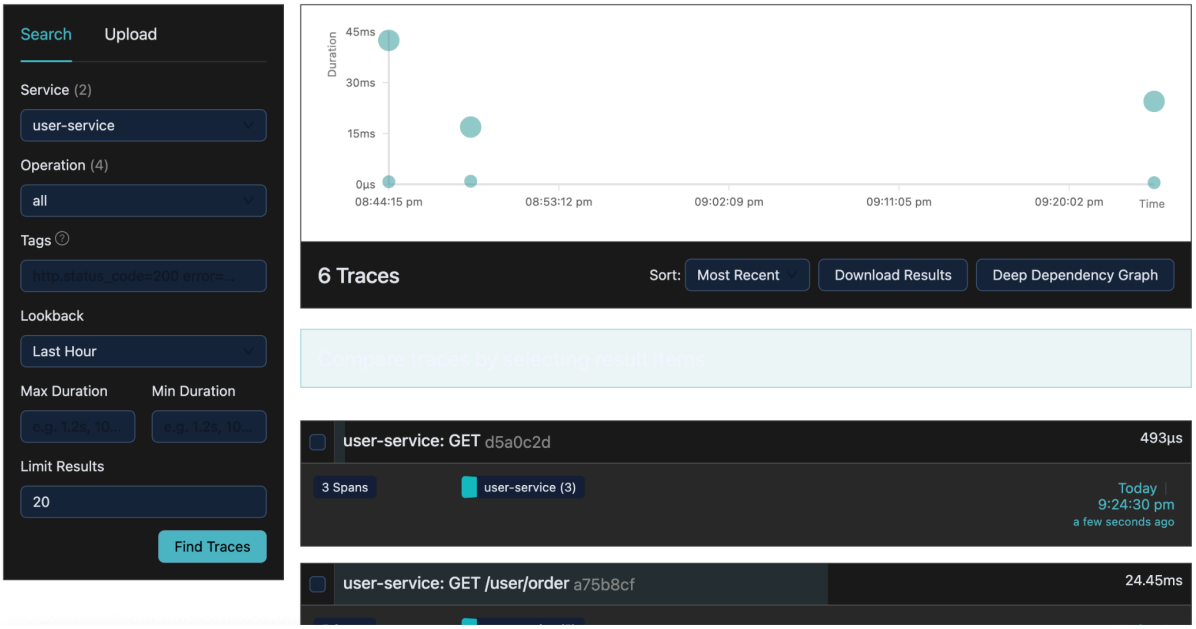
Protocol (OTLP), allowing request execution to be visualised through the Jaeger web interface.

The current system architecture is illustrated below.




```
(venv) maheendranpadmakumar@Maheendrans-MacBook-Air multi-agent-devops % kubectl get pods -n devops-ai
NAME                            READY   STATUS    RESTARTS        AGE
nginx-56c45fd5ff-9hbkv          1/1     Running   1 (3d12h ago)   8d
user-service-5c4448f647-f5lkn   1/1     Running   0               75s
(venv) maheendranpadmakumar@Maheendrans-MacBook-Air multi-agent-devops % docker images

IMAGE                                          ID             DISK USAGE   CONTENT SIZE   EXTRA
confluentinc/cp-kafka:7.5.0                    af34583c49f0        849MB             0B    U
confluentinc/cp-zookeeper:7.5.0                04bd40128a4e        849MB             0B    U
event-driven-ecommerce-api-gateway:latest      1169b78c8b90        263MB             0B    U
event-driven-ecommerce-eureka-server:latest    4a982757e4f3        264MB             0B    U
event-driven-ecommerce-inventory-service:latest c463bae9f349       307MB             0B    U
event-driven-ecommerce-notification-service:latest 698e75bf9e2d    310MB             0B    U
event-driven-ecommerce-order-service:latest    70b962648c8c        311MB             0B    U
event-driven-ecommerce-payment-service:latest  d3ebdacdbd0c        311MB             0B    U
event-driven-ecommerce-shipping-service:latest 4f308737ce04        309MB             0B    U
gcr.io/k8s-minikube/kicbase:v0.0.50            f3db27eba481       1.35GB             0B    U
ghcr.io/kafbat/kafka-ui:latest                 12e1f855cbcd        354MB             0B    U
hello-world:latest                             eb84fdc6f2a3        5.2kB             0B    U
postgres:15                                    aca5bef375a5        445MB             0B    U
sonarqube:lts-community                        e1ce274727b8        596MB             0B    U
user-service:v1                                fb5038994857        197MB             0B    U
(venv) maheendranpadmakumar@Maheendrans-MacBook-Air multi-agent-devops %
```



Challenges

Several technical challenges were encountered throughout the implementation. Initially, deploying Jaeger failed due to an incorrect Docker image tag, resulting in an ImagePullBackOff error. This issue was diagnosed using kubectl describe pod, which identified that the specified image manifest could not be located. Updating the deployment configuration with the correct image version resolved the problem successfully.

Another challenge involved configuring communication between Kubernetes services. Initially, service requests failed because of incorrect deployment configurations and image management within Minikube. These issues were resolved by rebuilding Docker images, loading them into Minikube, updating Kubernetes deployment manifests, and restarting the affected deployments.

The implementation of OpenTelemetry also required additional investigation to understand distributed tracing concepts, trace propagation, and exporter configuration. Through documentation review and practical experimentation, I successfully instrumented the User Service and verified trace generation within Jaeger.

Summary

The project has successfully established the foundation for an AI-assisted cloud-native observability platform. Functional microservices have been developed and deployed on Kubernetes, and distributed tracing has been implemented using OpenTelemetry and Jaeger. The next phase of the project will focus on instrumenting the remaining microservices, integrating Prometheus and Grafana for metrics

collection and visualisation, and developing a LangGraph-based multi-agent system capable of analysing logs, metrics, and traces to automate incident diagnosis.

References

[1] OpenTelemetry, OpenTelemetry Documentation.

[Online]. Available: https://opentelemetry.io/

[2] Jaeger Authors, Jaeger Documentation.

[Online]. Available: https://www.jaegertracing.io/

[3] Kubernetes Authors, Kubernetes Documentation.

[Online]. Available: https://kubernetes.io/docs/

[4] Docker Inc., Docker Documentation.

[Online]. Available: https://docs.docker.com/

[5] FastAPI, FastAPI Documentation.

[Online]. Available: https://fastapi.tiangolo.com/